Reducing Programs to Objects

Ver: 0.0.0

YEGOR BUGAYENKO, Huawei, Russia

C++, Java, C#, Python, Ruby, and JavaScript are the most powerful object-oriented programming languages if language power is defined as the number of *features* available to a programmer. This abundance of features makes their programs hard to reason about, to optimize, and to debug, and—unlike functional programming with its λ-calculus—object-oriented programming has no minimal foundation to which its programs can be lowered. EO, on the other hand, is an object-oriented programming language with a reduced set of features: it has nothing but objects and mechanisms of their composition and decoration. This paper asks the following research question: "Which known features are possible to implement using only objects?"

1 Introduction

Modern object-oriented languages are large and still growing. C++, Java, C#, Python, Ruby, and JavaScript each offer hundreds of features, and every new release adds more. The more features a language has, the harder its programs are to reason about, to optimize, and to search for bugs, because every feature may interact with every other one [? ? ? ?]. Functional programming rests on a small and well-understood foundation, the λ-calculus, into which its many features can be lowered and then analyzed uniformly. Object-oriented programming has no such foundation in common use: we know of no minimal object model to which an arbitrary object-oriented program can be lowered. EO was earlier proposed as exactly such a foundation [?], yet it was never demonstrated how programs written in Java, C++, or other mainstream languages map onto it. This paper fills that gap.

The work of ?] explains the design of the EO standard library of objects, which are used below.

2 Features

We selected the most complex features of the most popular object-oriented programming languages—C++, Java, C#, Python, Ruby, and JavaScript—and demonstrated how each of them may be represented in EO [?] objects[1]. Where a feature is best illustrated by another language, such as C, Kotlin, or PHP, we use that language instead.

Other features—such as operators, loops, generators, procedures, variables, code blocks, constants, and selection—are more trivial, which is why they are not presented in this paper.

---

[1] LaTeX sources of this paper are maintained in REPOSITORY GitHub repository, the rendered version is 0.0.0.

2.1 Static Methods

A static method (or static function) is a method defined as a member of a class but accessible directly through the class itself, rather than through an instance created by its constructor. For example, this C# class consists of a single static method. It may be converted to the following EO code, since a static method is nothing more than a "global" function.

```
class Utils {
  public static int max(int a, int b) {
    if (a > b) return a;
    return b;
  }
}
```

```
if. > [a b] > max
  a.gt b
  a
  b
```

2.2 Goto

A "backward" jump in the C language can be mapped to EO recursion:

```
#include "stdio.h"
void f() {
  int i = 1;
  again:
  i++;
  if (i < 10)
    goto again;
  printf("Finished!");
}
```

```
malloc.empty > [] > f
  seq * > [i]
    i.put 1
    again i
    io.stdout "Finished!"
  seq * > [ii] > again
    ii.put (ii.as-number.plus 1)
    if.
      ii.as-number.lt 10
      again ii
      true
```

A "forward" jump in the C language can be mapped to EO:

```
int f(int x) {
  int r = 0;
  if (x == 0)
    goto exit;
  r = 42 / x;
  exit:
  return r;
}
```

```
malloc.empty > [x] > f
  seq * > [r] >>
    r.put 0
    if.
      x.eq 0
      true
      r.put (42.div x)
    number r.get
```

Here, the jump to the `exit` label becomes a conditional that skips the division when `x` is zero, so `r` keeps its initial value.

According to the Böhm–Jacopini theorem [?], any use of the `GOTO` operator may be replaced by a combination of sequence (the `seq` object), selection (the `if` attribute), and iteration (the `while` object). Relying on this fact, we may conclude that any C code that uses `GOTO` may be represented in EO.

2.3 Pointers

This is an example of C code, where the pointer `p` is first incremented by seven times `sizeof(book)` (which is equal to 104) and then de-referenced to become a struct `book` mapped to memory. Then, the `title` part of the struct is filled with a string and the `price` part is returned as a result of the function `f`. The algorithm can be represented in EO on top of the `malloc` object.

```
#include "string.h"
struct book {
  char title[100];
  int price;
};
int f(struct book* p) {
  struct book b = *(p + 7);
  strcpy(b.title, "1984");
  return b.price;
}
```

```
[mem offset] > book
  book > [i] > plus
    mem
    offset.plus (i.times 104)
  as-i32. > get-price
    mem.read (offset.plus 100) 4
  mem.write offset s > [s] > set-title
[p] > f
  (book p).plus 7 > b
  seq * > @
    b.set-title "1984"
    b.price
```

A pointer may not only refer to data in the heap but also to executable code in memory (there is no difference between "program" and "data" memory in both x86 and ARM architectures). This is an example of a C program that calls a function referenced by a function pointer, providing two integer arguments to it. It can easily be mapped to EO via its partial application:

```
typedef int foo(int, int);
int f(foo *p) {
  return (*p)(7, 42);
}
```

```
[x y] > foo

[foo] > f
  foo 7 42
```

It is important to note that the following code cannot be represented in EO:

```
int f() {
  int (*p)(int);
  p = (int(*)(int)) 0x761AFE65;
  return (*p) (42);
}
```

Here, the pointer refers to an arbitrary address in program memory, where some executable code is supposed to be located.

This C function, which returns seven (tested with GCC), assumes that variables are placed in the stack sequentially and uses pointer de-referencing to access them.

```
long f() {
  long a = 42;
  long b = 7;
  return *(&a - 1);
}
```

```
malloc.empty > [] > f
  seq * > [m]
    m.write 0 7
    m.write 8 42
    number
      m.read (8.minus 8) 8
```

2.4 Impure Functions

A function is pure when, among other qualities, it does not have side effects: no mutation of local static variables, non-local variables, mutable reference arguments, or input/output streams. This impure PHP function can be mapped to EO:

```
$a = 42;
function inc() {
  $a = $a + 1;
  return $a;
}
inc(inc($a)); // $a == 44
```

```
[a] > inc
  number a.get > ia
  seq * > @
    a.put (ia.plus 1)
    ia
```

Here, `a` is an abstraction of a memory block. It must be passed as an argument to the `inc` object. In PHP, and many other languages, the heap is available in global state, while in EO it must be passed explicitly as an argument.

2.5 Closures

A closure is a function bundled with the variables it captured from its defining scope. This JavaScript function returns a counter that increments a captured variable on each call.

```
function counter() {
  let c = 0;
  return () => ++c;
}
var next = counter();
next(); // 1
next(); // 2
```

```
[c] > counter
  seq * > [] > next
    c.put (c.as-number.plus 1)
    c.as-number
malloc.empty > [] > app
  [c]
    counter c > cnt
    seq * > @
      c.put 0
      cnt.next
      cnt.next
```

In EO, a closure is just an object, and the captured variable, being mutable, is a `malloc` block. The only open question is ownership. The block cannot belong to the closure `next` itself, nor to the `counter` that builds it, because EO memory is scoped and would be released once the maker returns. It must be owned by an object that outlives the closure—here the `app` object—which allocates the block and passes it in.

As with impure functions, EO grants no implicit access to captured state: it must be threaded through explicitly.

2.6 Classes

A class is an extensible program code template for creating objects, providing initial values for state (member variables) and implementations of behavior (member functions or methods). The following program contains a Ruby class with a single constructor, two methods, and one mutable member variable. It can be mapped to the following EO code, where a class becomes an object factory.

```
class Book
  def initialize(i)
    @id = i
    puts "New book!"
  end
  def path
    "/tmp/${@id}.txt"
  end
  def move(i)
    @id = i
  end
end
```

```
[id i] > book
  seq * > [] > initialize!
    id.put i
    io.stdout "New book!"
  seq * > [] > path
    initialize
    tt.sprintf *1
      "/tmp/%s.txt"
      id
  seq * > [i] > move
    initialize
    id.put i
```

Here, the constructor is represented by the object `initialize`, whose `!` suffix makes it a constant, so it runs only once no matter how many methods call it. Making an “instance” of a book would look like this:

```
malloc.empty
  [id]
    book id 42 > b
    seq * > @
      b.move 7
      b.path
```

2.7 Encapsulation

Encapsulation hides the internal state and helper behavior of an object behind a public interface. In this Java class, the field `rate` and the method `tax` are private, while only `total` is exposed to callers.

```
class Cart {
  private int rate = 10;
  private int tax(int p) {
    return p / rate; }
  int total(int p) {
    return p + tax(p); } }
```

```
[] > cart
  p.plus > [p] > total
    p.div 10
```

EO has no `private` or `protected` modifiers: an object either names an attribute or it does not. Here, both `rate` and `tax` dissolve into the anonymous sub-expression `p.div 10`, which, having no name, cannot be reached from outside, so their hiding is total. Data hiding is thus achieved by anonymity: whatever is not named cannot be addressed. When a private member must keep a name, such as a mutable field shared between methods, its privacy reduces to a naming convention, since objects carry no notion of visibility.

2.8 Static Fields

A static field is mutable state shared by every instance of a class. In this Java class, the counter `count` is incremented by the constructor, so it holds the number of books ever created.

```
class Book {
  static int count = 0;
  Book() {
    count += 1;
  }
}
```

```
[count] > book
  count.put > [] > make
    count.as-number.plus 1
malloc.empty > [] > app
  [count]
    seq * > @
      count.put 0
      (book count).make
      (book count).make
      count.as-number
```

EO has no class to hold shared state, so the `count` block must be owned by an object that outlives every instance: the application object `app` itself. Each `book` receives the same block, and an increment made through one instance is visible through all of them. What other languages hide behind the word `static` becomes explicit—the owner of class-level state is simply the top-level object of the program.

2.9 Destructors

In C++ and some other languages, a destructor is a method that is called for a class object when that object passes out of scope or is explicitly deleted. The following code prints both `Alive` and `Dead` texts. In EO, the destructor must be called explicitly by the owner of the object.

```
#include <iostream>
class Foo {
public:
  Foo() { std::cout << "Alive"; }
  ~Foo() { std::cout << "Dead"; };
};
int main() {
  Foo f = Foo();
}
```

```
[] > foo
  io.stdout "Alive" > [] > ctor
  io.stdout "Dead" > [] > dtor
[] > main
  foo > f
  seq * > @
    f.ctor
    f.dtor
```

2.10 Exceptions

This section is not valid, must be rewritten!

Exception handling is the process of responding to the occurrence of exceptions: anomalous or exceptional conditions requiring special processing—during the execution of a program. This C++ program utilizes exception handling to avoid segmentation fault due to null pointer de-referencing.

```
#include <iostream>
int price(int book) {
  if (book == 42) throw "Nope!";
  return 7;
}
void print(int b) {
  try {
    std::cout << price(b);
  } catch (char const* e) {
    std::cout << "Error: " << e;
  }
}
```

```
if. > [book cant-return] > price
  book.eq 42
  cant-return "Nope!"
  7
seq * > [b] > print
  io.stdout
    price
      b
      [m]:cant-return
```

A Java method may throw a number of either checked or unchecked exceptions:

```
void f(int x) throws Exception {
  if (x == 0)
    throw new IOException("foo");
  throw new Exception("bar");
}
void bar() {
  try {
    f(42);
  } catch (IOException e) {
    echo(e.getMessage());
  } catch (Exception e) {
    System.out.exit(1);
  }
}
```

```
if. > [x] > f
  x.eq 0
  T "foo"
  T "bar"
```

2.11 Types and Type Casting

A type system is a logical system comprising a set of rules that assign a property called a type to various constructs of a computer program, such as variables, expressions, functions, or modules. The main purpose of a type system is to reduce the possibility of bugs in computer programs by defining interfaces between different parts of a computer program, and then checking that the parts have been connected in a consistent way. In this example, a Java method `cost` expects an argument of type `Book` and compilation would fail if another type is provided.

```
int cost(Book b) {
  return 42 + b.price();
}
```

The restrictions enforced by the Java type system at compile time through the `Book` type are enforced in EO by automatic type inference.

2.12 Reflection

Reflection is the ability of a process to examine, introspect, and modify its own structure and behavior. In the following Python example, the method `hello` of an instance of class `Foo` is not called directly on the object but is first retrieved through reflection functions `globals` and `getattr`, and then executed:

```
def Foo():
    def hello(self, name):
        print("Hello, %s!" % name)
obj = globals()["Foo"]()
getattr(obj, "hello")("Jeff")
```

It may be implemented in EO by creating a global dictionary of classes and their methods and filling it up at compile time. Also, all objects may be equipped with a mechanism of registration in the dictionary, which will make them discoverable by reflection.

2.13 Monkey Patching

Monkey patching is making changes to a module or class while the program is running. This JavaScript program adds a new method `print` to the object `b` after the object has already been instantiated:

```
function Book(t) { this.title = t; }
var b = new Book("Object Thinking");
b.print = function() {
  console.log(this.title);
}
b.print();
```

This program may be translated to EO by introducing a new `Book1` class with the `print` method defined differently.

2.14 Inheritance

Inheritance is the mechanism of basing a class upon another class while retaining similar implementation. In this Java code class `Book` inherits all methods and non-private attributes from class `Item`. In EO, composition may be used instead of inheritance.

```
class Item {
  private int p;
  int price() { return p; }
}
class Book extends Item {
  int tax() {
    return price() * 0.1;
  }
}
```

```
[p] > item
  p > [] > price
[item] > book
  item.price.times 0.1 > [] > tax
```

So-called prototype-based programming uses generalized objects, which can then be cloned and extended. For example, this JavaScript program defines two objects, where `Item` is the parent object and `Book` is the child that inherits the parent through its prototype.

```
function Item(p) {
  this.price = p;
}
function Book(p) {
  Item.call(this, p);
  this.tax = function () {
    return this.price * 0.1;
  }
}
var t = new Book(42).tax();
console.log(t); // prints "4.2"
```

```
[p] > item
  p > price
[item] > book
  item > @
  ^.price.times 0.1 > [] > tax
(book 42).tax > t
io.stdout (as-fixed t)
```

Multiple inheritance is a feature of some object-oriented programming languages in which an object or class can inherit features from more than one parent object or parent class. In this C++ example, the class `Jack` has both `bark` and `listen` methods, inherited from `Dog` and `Friend` respectively.

```
#include <iostream>
class Dog {
  virtual void bark() {
    std::cout << "Bark!";
  }
};
class Friend {
  virtual void listen();
};
class Jack: Dog, Friend {
  void listen() override {
    Dog::bark();
    std::cout << "Listen!";
  }
};
```

```
[] > dog
  io.stdout "Bark!" > [] > bark
[] > friend
  [] > listen
[] > jack
  dog > d
  friend > f
  seq * > [] > listen
    d.bark
    io.stdout "Listen!"
```

Here, inherited methods are explicitly listed as attributes in the object `jack`. This is very close to what would happen in the virtual table of a class `Jack` in C++. The EO object `jack` just makes it explicit.

2.15 Polymorphism

Subtype polymorphism lets a single call site invoke different implementations depending on the runtime type of the receiver, a mechanism known as dynamic dispatch. In this Java program, `speak` accepts any `Animal`, yet the sound it prints is chosen by the concrete object passed to it.

```
class Animal {
  String sound() {
    return "?";
  }
}
class Dog extends Animal {
  String sound() {
    return "Bark";
  }
}
void speak(Animal a) {
  print(a.sound());
}
speak(new Dog());
```

```
[] > animal
  "?" > [] > sound
[] > dog
  animal > @
  "Bark" > [] > sound
[a] > speak
  io.stdout a.sound
speak dog
```

In EO, a method call is nothing but attribute resolution on the receiver, so dispatch needs no special mechanism: `a.sound` always finds the `sound` of the object bound to `a`. The object `dog` decorates `animal` through `@`, yet defines its own `sound`, which shadows the inherited one. No virtual table is needed, because each object carries its own behavior.

2.16 Abstract Classes and Interfaces

An interface, or an abstract class, declares methods without implementing them, leaving the implementation to conforming types. In this Java example, `Shape` declares `area` and `Square` implements it.

```
interface Shape {
  int area();
}
class Square implements Shape {
  int side;
  public int area() {
    return side * side;
  }
}
```

```
[] > shape
  [] > area
[side] > square
  side.times side > [] > area
```

In EO, the interface `shape` is an abstract object whose `area` attribute has no body; such an object is incomplete and serves purely as a contract. `square` completes the contract by binding `area`. No `implements` keyword is required: any object that provides the named attributes satisfies the interface, and wherever a `shape` is expected, type inference verifies that `area` is bound. An abstract class is the same idea, with some attributes already implemented and others left void.

2.17 Method Overloading

Method overloading is the ability to create multiple functions with the same name but different implementations. Calls to an overloaded function will run a specific implementation of that function appropriate to the context of the call, allowing one function call to perform different tasks depending on context. In this Kotlin program, two functions share the same name, and the call `foo(42)` resolves to the one that takes an integer.

```
fun foo(a: Int) {}
fun foo(a: Double) {}
foo(42)
```

```
[a] > foo-with-int
[b] > foo-with-double
foo-with-int 42
```

EO has no overloading, so each variant becomes a distinct object with a unique name, and the call selects the one whose name matches the argument type.

2.18 Java Generics

Generics extend Java's type system to allow a type or method to operate on objects of various types while providing compile-time type safety. For example, this Java class expects another class to be specified as `T` before use:

```
class Cart<T extends Item> {
  private int total;
  void add(T i) {
    total += i.price();
  }
}
```

```
[total] > cart
  number total.get > t
  total.put > [i] > add
    t.plus i.price
```

The type parameter `T` disappears: `cart` accepts any object that understands `price`, while type inference recovers the compile-time safety that Java enforced through `T`.

2.19 C++ Templates

Templates are a feature of the C++ programming language that lets functions and classes operate with generic types, working with many different data types without being rewritten for each one. For example, this function returns the larger of two arguments of any comparable type `T`:

```
template<typename T> T max(T a, T b) {
  return a > b ? a : b;
}
int x = max(7, 42);
```

```
[a b] > max
  if. > @
    a.gt b
    a
    b
max 7 42 > x
```

The EO `max` needs no type parameter: the same object works for any argument that understands `gt`.

2.20 Mixins

A mixin is a class that contains methods for use by other classes without having to be the parent class of those other classes. The following code demonstrates how a Ruby module is included in a class:

```
module Timing
  def recent?
    @time - Time.now < 24 * 60 * 60
  end
end
def News
  include Timing
  def initialize(t)
    @time = t
  end
end
n = News.new(Time.now)
n.recent?
```

This code may be represented in EO by just copying the method `recent?` to the object `News` as if it were defined there.

2.21 Annotations

In Java, an annotation is a form of syntactic metadata that can be added to source code; classes, methods, variables, parameters, and Java packages may be annotated. Later, annotations may be retrieved through the Reflection API. For example, this program analyzes the annotation attached to the class of the provided object and changes its behavior depending on one of the annotation's attributes.

```
interface Item {}
@Ship(true)
class Book implements Item {}
@Ship(false)
class Song implements Item {}
class Cart {
  void add(Item i) {
    /* add the item to the cart */
    if (i.getClass()
      .getAnnotation(Ship.class)
      .value()) {
      /* mark cart as shippable */
    }
  }
}
```

```
[] > book
  ship true > a1
[] > song
  ship false > a1
[] > cart
  if. > [i] > add
    i.a1.value
      true
      false
```

Here, each annotation becomes an attribute of the object: `a1` holds the `ship` object, and `cart` reads `i.a1.value` to choose its behavior, just as the Java code reads it through reflection.

2.22 Concurrency

This section is not valid, must be rewritten!

Concurrency lets multiple computations run simultaneously, leaving the language to coordinate their access to shared memory and their order of execution. This Java program starts two threads that print concurrently, so the order of the output is not determined in advance.

```
new Thread(() ->
  System.out.print("A")
).start();
new Thread(() ->
  System.out.print("B")
).start();
```

EO does not yet define a model for concurrent or parallel execution: it has no notion of a thread, a lock, or a scheduler. Until such a model is designed, programs that rely on nondeterministic concurrent execution cannot be represented as EO objects.

2.23 Traceability

A certain amount of semantic information may be lost during the translation from a more powerful programming language to EO objects, such as, for example, names, line numbers, comments, etc. Moreover, it may be useful to have the ability to trace EO objects back to the original language constructs from which they were derived. For example, a simple C function can be mapped to the following EO object.

```
int f(int x) {
  return 42 / x;
}
```

```
[x] > f
  [] > @
    "src/main.c:1-1" > source
    42.div x > @
  "src/main.c:0-2" > source
```

Here, the synthetic `source` attribute represents the location in the source code file with the original C code. It is important to ensure during translation that the name `source` does not conflict with a possibly similar name in the object.

3 Related Work

The conviction that a small, primitive foundation can express any computation is older than programming itself. The Turing machine [?] was perhaps the first such idea: a device with a handful of fixed operations—read, write, move, and change state—that is nonetheless able to compute anything computable. Church's λ-calculus [? ?] reached the same conclusion from another direction, reducing all computation to the formation and application of functions, while combinatory logic [?] went further and dispensed with variables altogether, leaving only a fixed basis of combinators. Each showed that vast expressive power can rest on very few primitives. Our work is driven by the same conviction, narrowed to object-oriented programming: that the many features of a mainstream language rest, ultimately, on the single primitive of the object.

The same instinct reappears in the design of languages and machines. ?] proved that any control flow reduces to sequence, selection, and iteration, the argument we reuse in Section 2.2 to eliminate `goto`. ?] framed the surface syntax of a language as "syntactic sugar" over a small applicative kernel, and ?] re-expressed imperative constructs—assignment, loops, and jumps—as local transformations into the λ-calculus, feature by feature, much as we do here for objects. At the level of hardware, the one-instruction computer [?] reduces an entire processor to a single operation.

A closely related line of work defines a full, real-world language by translating it into a small core. ?] desugar JavaScript into λ_{JS}, and ?] do the same for Python, each reducing a sprawling language to a compact calculus with a tested operational semantics. ?] organize an entire textbook around a "kernel language" into which every paradigm is translated one concept at a time. These efforts reduce a language to a λ-based or imperative core and aim at a precise, tested semantics, whereas we reduce to objects alone and give an informal, feature-by-feature mapping intended for practitioners.

The reduction of objects themselves to a minimal core has been studied formally. **?**] distill Java to Featherweight Java, a calculus small enough to fit a soundness proof on a single page, yet one that keeps classes, inheritance, and types as primitives. **?**] build objects directly, with no λ-calculus underneath, as the primitives of their object calculus, while others instead encode objects as records of functions and inheritance as self-referential modification [**? ?**], an approach rooted in the older duality between objects and abstract data types [**?**]. The prototype-based language Self [**?**] pursued the same minimalist instinct at the level of language design, discarding classes in favor of objects and cloning. Our work rests on φ-calculus [**? ?**], the formalism behind EO, but differs from all of the above in direction: rather than encoding objects inside a smaller calculus, we take objects as given and show that the features of mainstream languages fold into them.

4 Conclusion

We demonstrated how some language features often present in high-level object-oriented languages, such as Java or C++, may be expressed using objects. We used EO as a destination programming language because of its minimalistic semantics: it has no language features aside from objects and their composition and decoration mechanisms.